

GenAI Project: Intelligent PDF Q&A Bot with RAG using LangChain

1. What is RAG (Retrieval Augmented Generation)?

RAG is a GenAI architecture where an LLM is enhanced with external data sources. Instead of relying only on its training data, the LLM can access and use additional documents (like PDFs, wikis, or databases) to answer questions.

It works in two stages:

- Retrieval: The system fetches relevant pieces of external content based on your query.
- Generation: It sends both your query and the retrieved content to the LLM to generate a final, accurate response.

2. Why We Use RAG in This Project

Our goal is to allow users to ask questions about the content of a PDF file. Since LLMs like GPT-3.5 don't "remember" uploaded files on their own, we use RAG to:

- Store the PDF's knowledge in a retrievable format (vector embeddings)
- Retrieve only relevant parts when the user asks a question
- Provide precise, grounded answers

This allows us to build a Q&A system without fine-tuning the model.

3. Project Flow

1. Load a PDF
2. Split its text into smaller chunks
3. Convert each chunk to vector using OpenAI Embeddings
4. Store those vectors in FAISS (a fast vector search DB)
5. When a user asks a question:
 - It's embedded
 - Relevant chunks are retrieved using similarity search

GenAI Project: Intelligent PDF Q&A Bot with RAG using LangChain

- LLM uses them to generate an answer

This is a textbook application of RAG: retrieve + generate.

4. Code Skeleton (Overview)

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# Load PDF and split text
loader = PyPDFLoader("sample.pdf")
pages = loader.load_and_split()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
texts = splitter.split_documents(pages)

# Convert to embeddings and store in FAISS
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(texts, embeddings)

# Create retriever and QA chain
retriever = vectorstore.as_retriever()
llm = ChatOpenAI(model_name="gpt-3.5-turbo")
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)

# Ask questions
response = qa_chain.run("What is this PDF about?")
print(response)
```

GenAI Project: Intelligent PDF Q&A Bot with RAG using LangChain

5. Real-Life Use Cases

- Legal AI assistants (chat with contracts)
- Research paper summarizers
- Employee handbooks turned into bots
- Chat with product manuals
- AI study partner from books or notes

6. Final Thoughts

This project teaches not only LangChain and OpenAI integration but also how to structure real-world GenAI apps using RAG.

You can expand this by:

- Accepting multiple PDFs
- Creating a chat-like interface with Streamlit or Gradio
- Saving user history, adding citations, or generating summaries

With RAG, your LLM becomes an assistant who can read.